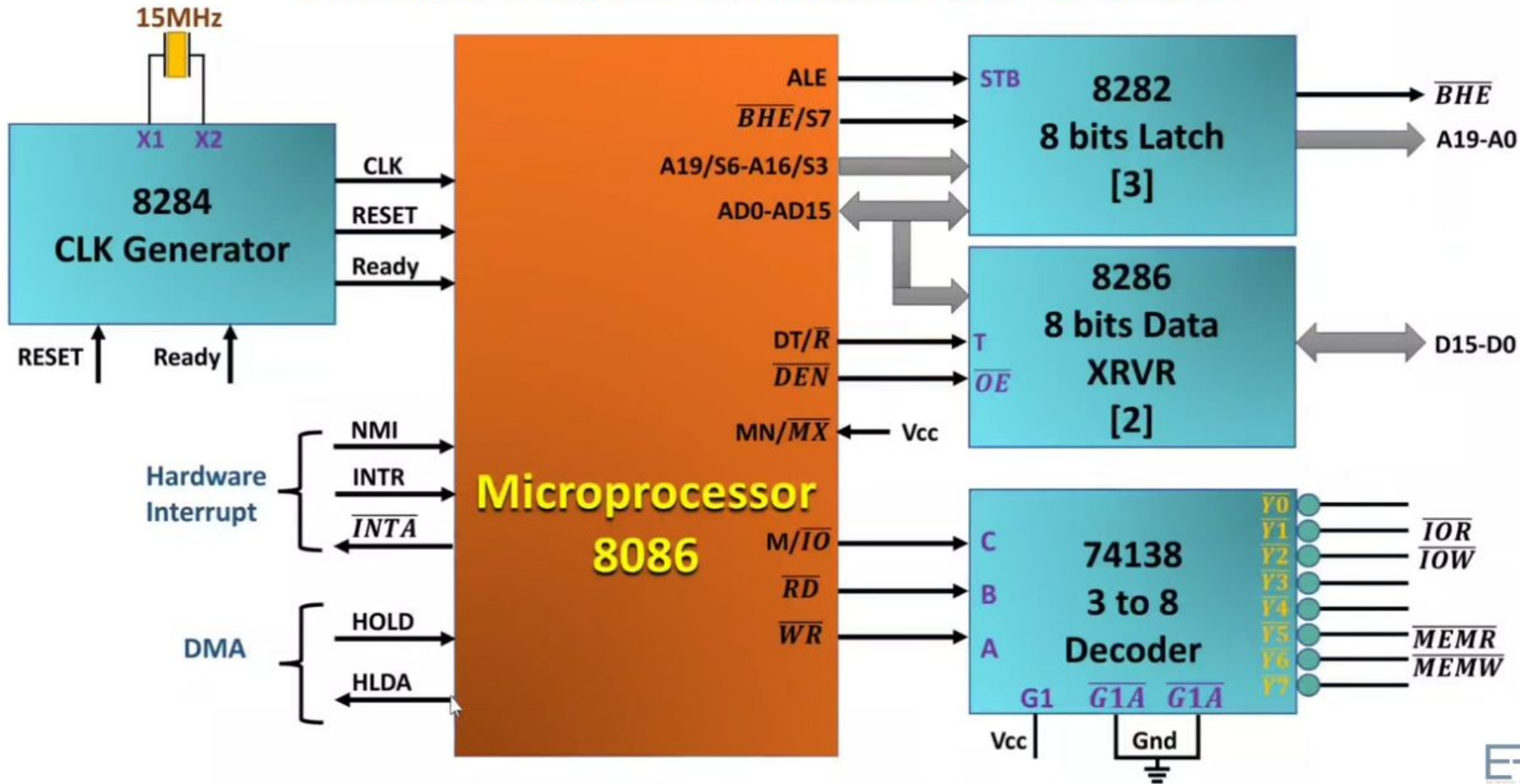


Microprocessors

Minimum Mode of 8086

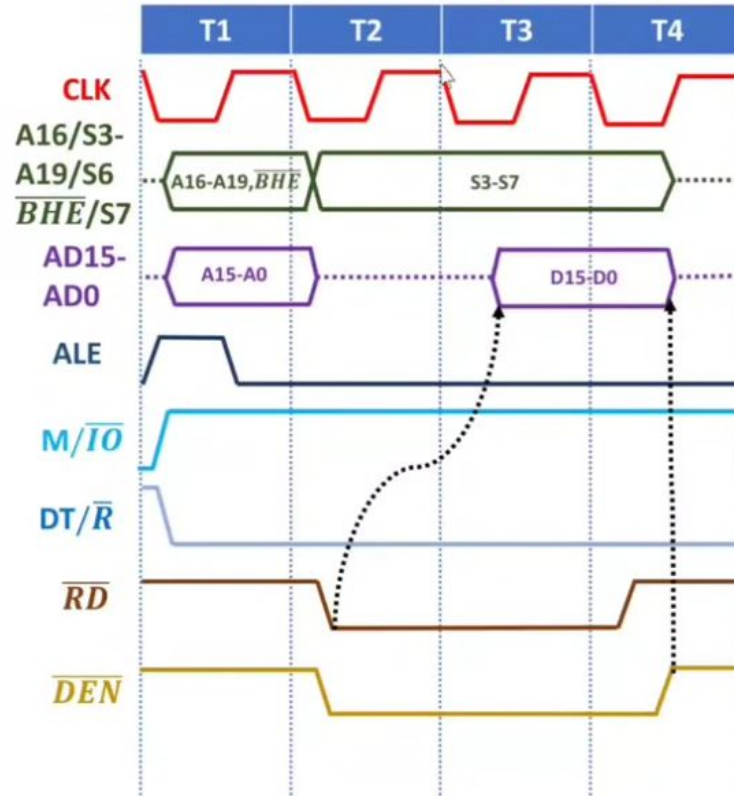


Timing Diagram

□ By using three terminals [$\overline{RD}$, $\overline{WR}$ & $M/\overline{IO}$], we can execute four Machine Cycles.

- Memory Read [$\overline{RD} = 0$, $\overline{WR} = 1$ & $M/\overline{IO} = 1$]
- Memory Write [$\overline{RD} = 1$, $\overline{WR} = 0$ & $M/\overline{IO} = 1$]
- IO Read [$\overline{RD} = 0$, $\overline{WR} = 1$ & $M/\overline{IO} = 0$]
- IO Write [$\overline{RD} = 1$, $\overline{WR} = 0$ & $M/\overline{IO} = 0$]

❖ Memory Read Timing Diagram



Question 1: Minimum Mode Memory Read Cycle

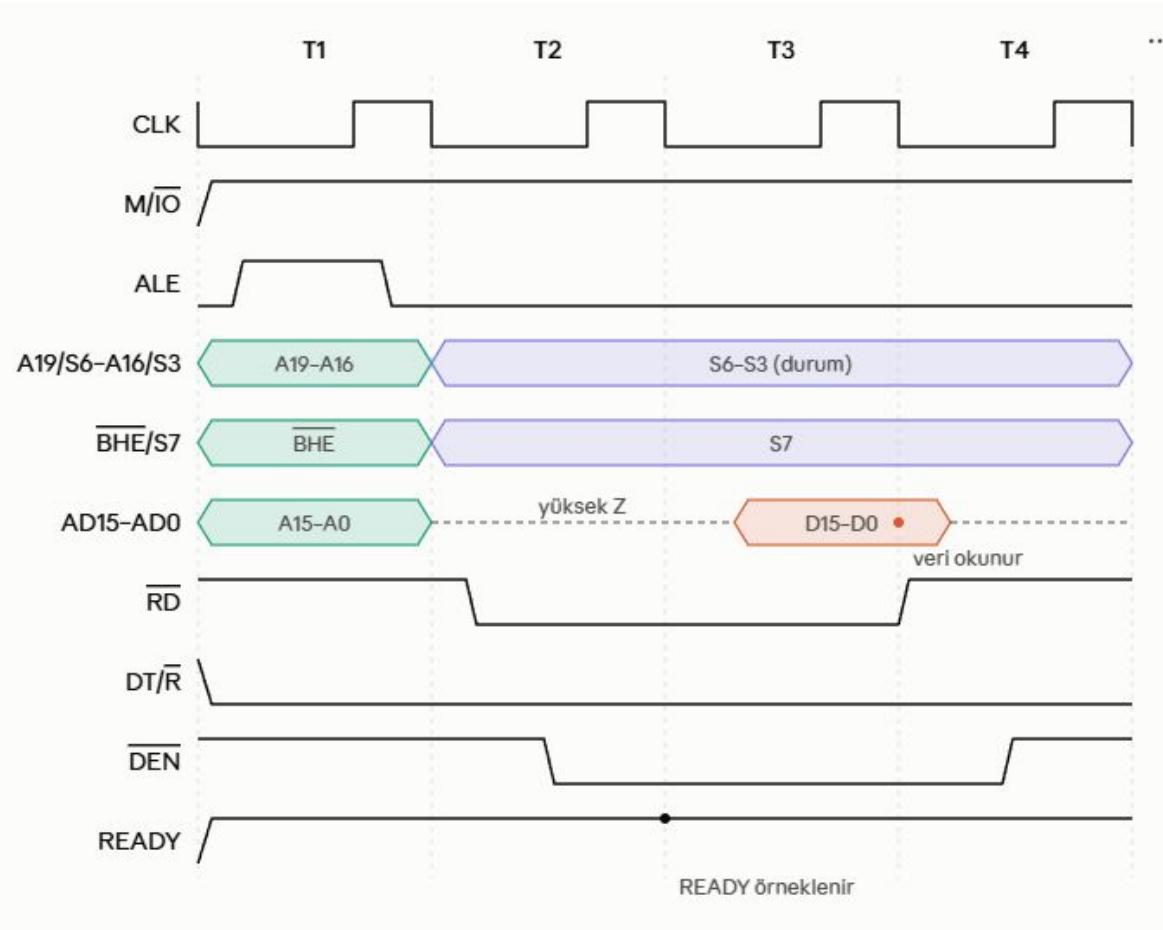
Task: Draw the timing diagram for a **Memory Read Cycle** in 8086 Minimum Mode across four clock periods (T_1 , T_2 , T_3 , T_4).

T1 – Address phase: $M/\overline{IO}$ goes high (memory access). The 20-bit address is placed on A19–A16 and AD15–AD0, and $BH\overline{E}$ is output. ALE pulses high, and external latches capture the address on its falling edge. $DT/\overline{R}$ goes low to set the transceivers to receive.

T2 – Bus turnaround: AD15–AD0 float (high-Z) to switch from address output to data input. The upper lines now carry status (S3–S6, S7). $\overline{RD}$ goes low to start the read, and $DE\overline{N}$ goes low to enable the data transceivers.

T3 – Data phase: READY is sampled. If it is low, wait states (T_w) are inserted. Memory places data on AD15–AD0.

T4 – Data read and end: The CPU reads the data at the start of T4. $\overline{RD}$ and $DE\overline{N}$ return high, and the bus is released for the next cycle.



Question 2: Memory Write Cycle Analysis

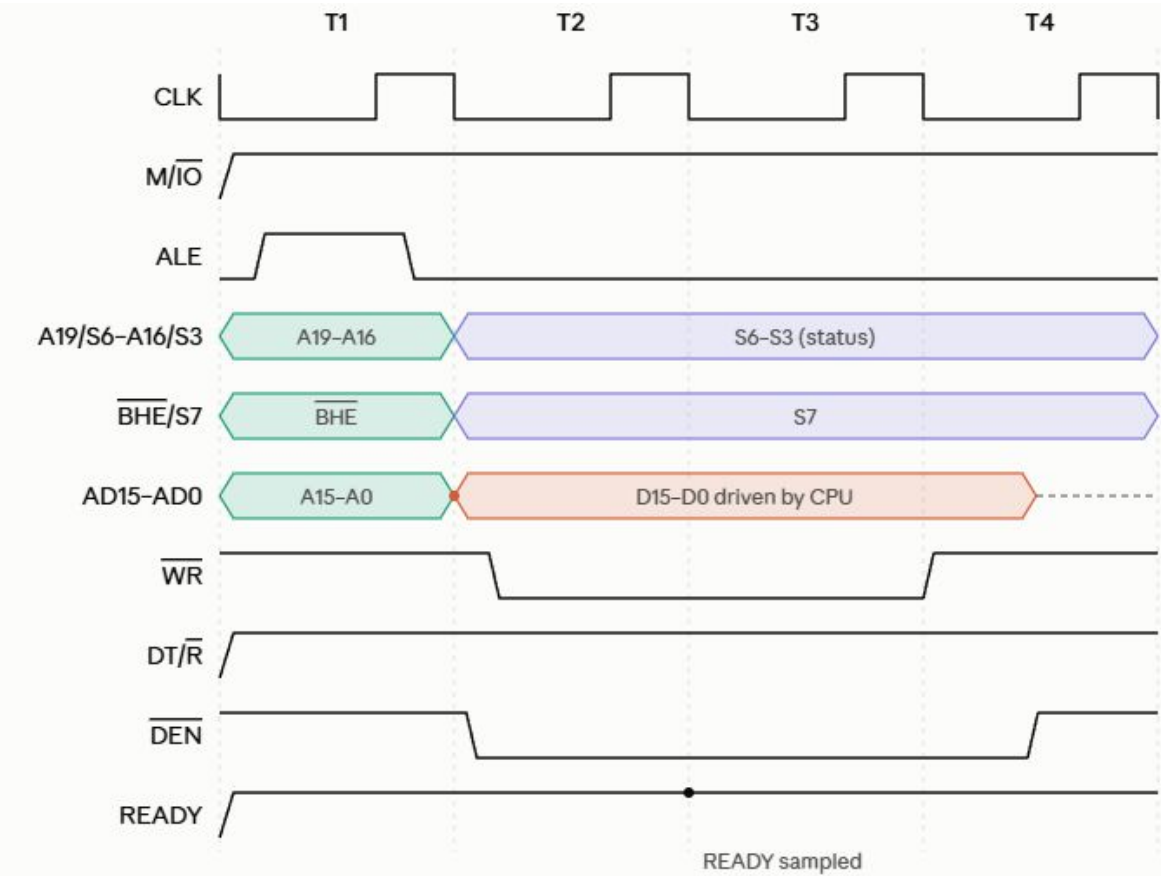
Task: Analyze the **Memory Write Cycle**. Explain how the $AD_{15}-AD_0$ bus behavior differs from a Read Cycle. Specifically, when does the processor start driving the **Data** onto the bus?

T1 is identical to the read cycle.

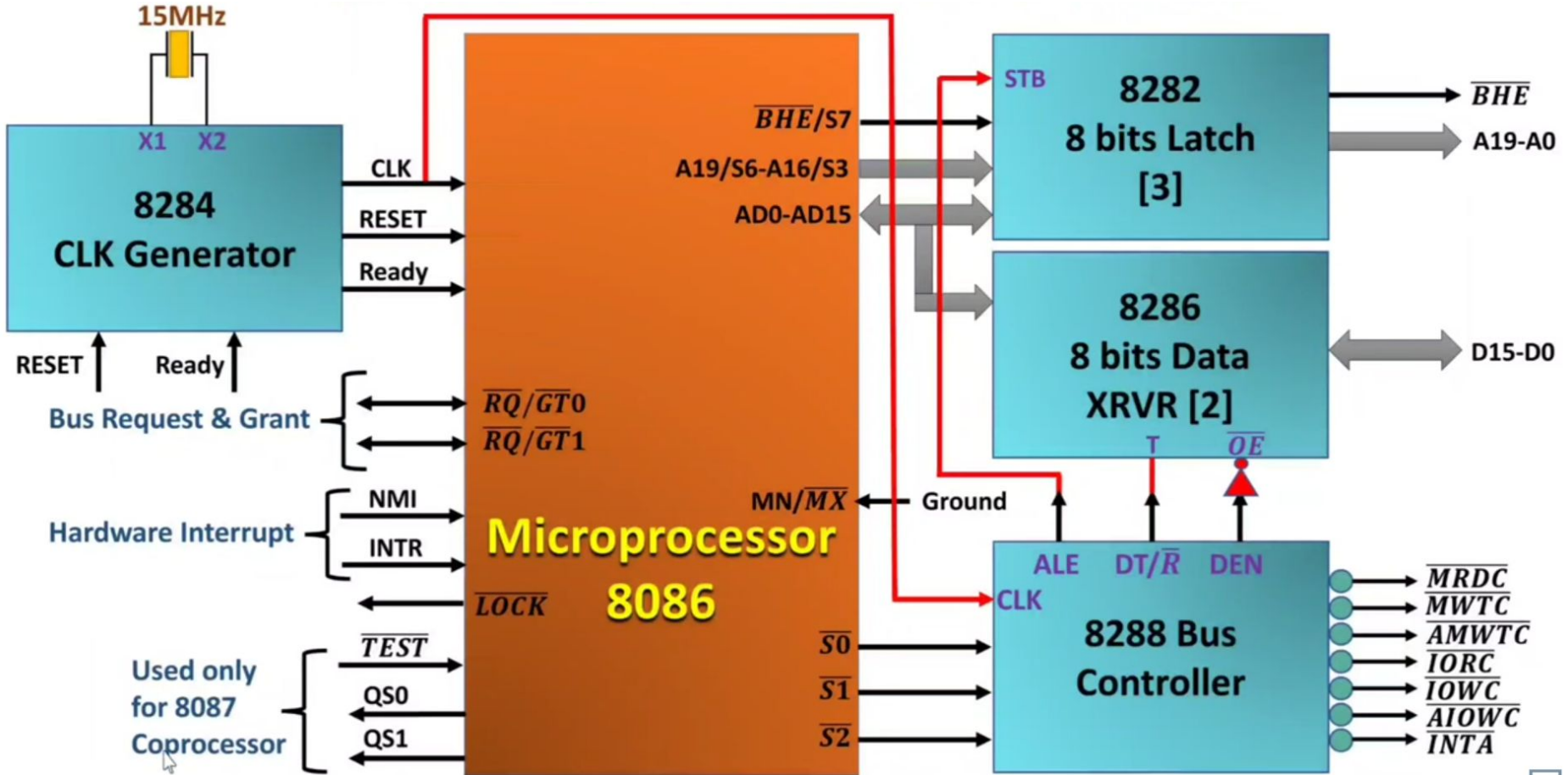
In T2, $\overline{WR}$ goes low and $\overline{DEN}$ goes low to enable the transceivers. The upper lines switch to status ($S3-S7$), as in a read.

In T3, $READY$ is sampled, and wait states (T_w) are inserted if it is low. Memory uses this time to store the data.

In T4, $\overline{WR}$ returns high. The rising edge of $\overline{WR}$ is typically where memory latches the data. The CPU keeps the data on the bus a little longer (hold time), then releases the bus and $\overline{DEN}$ goes high.



Maximum mode in 8086



Maximum mode in 8086 - Status signals with 8288 Bus Controller

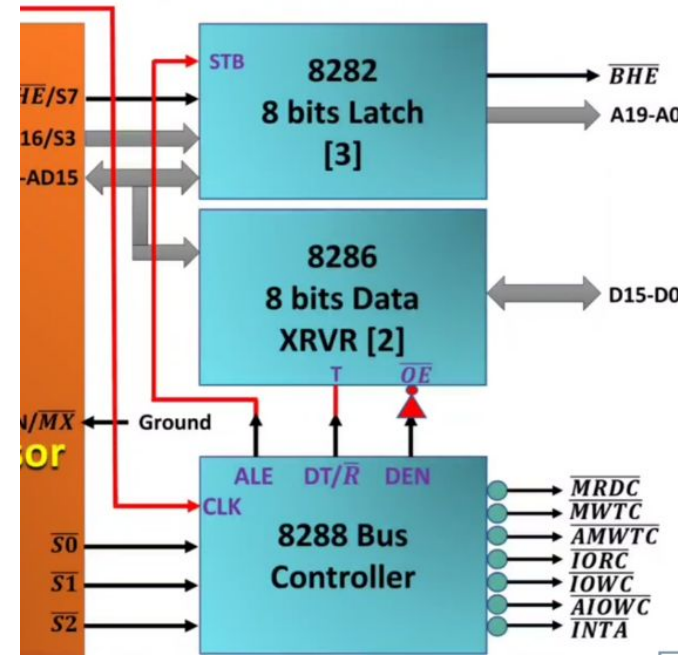
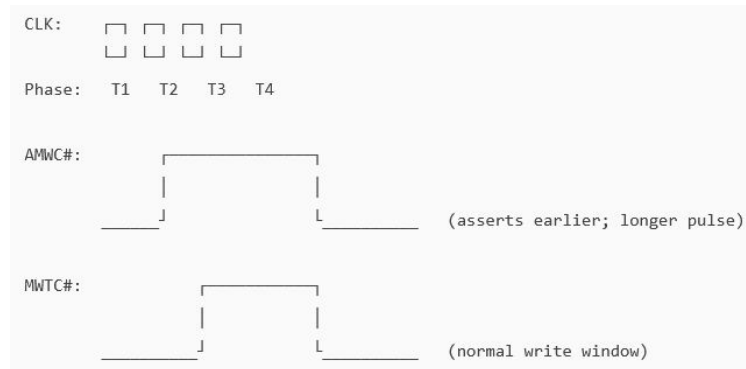
- In **Minimum Mode**, the 8086 processor handles all necessary control signals directly and does not require an additional bus controller.
- In **Maximum Mode**, the 8086 processor generates various **status signals** and these signals are received and processed by a bus controller such as the 8288.

The **8288 Bus Controller** receives these status signals and is used to control communication between the processor and other system components. This is especially important in systems with multiple processors or I/O devices.

Note:

MWTC (Memory Write Command): the *normal* active-low write command asserted during a write bus cycle to tell memory to store the data on the data bus.

AMWC (AMWTC - Advanced Memory Write Command): an *early* (advanced) active-low write command that goes active **one clock earlier** (timing aligned like a read command) to provide a **wider write pulse** for slower memory devices.



Question 3: Maximum Mode Memory Read Cycle

Task: Draw the timing diagram for a **Memory Read Cycle** in 8086 Maximum Mode across four clock periods (T_1 , T_2 , T_3 , T_4).

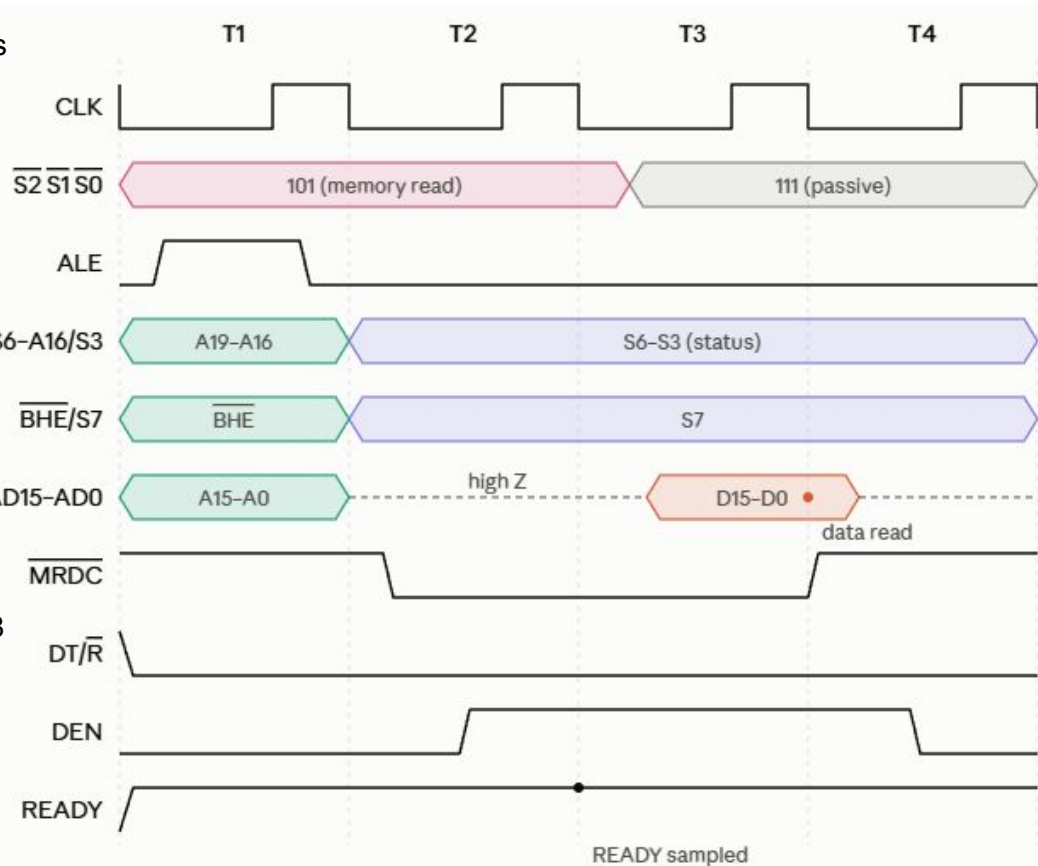
Before T_1 : The CPU changes $S_2\overline{S_1}\overline{S_0}$ from the passive state (111) to the memory read code (101). This change actually begins during T_4 of the previous cycle, so the 8288 knows in advance what kind of cycle is coming.

T_1 – Address phase: The 8288 detects the new status code and generates the ALE pulse. The CPU places the 20-bit address on A19–A16 and AD15–AD0, together with $\overline{BHE}$, and the latches capture it. The 8288 sets $DT/\overline{R}$ low (receive).

T_2 – Bus turnaround: AD15–AD0 float (high-Z), as in minimum mode. The upper lines switch to status (S_3 – S_7). The 8288 drives $\overline{MRDC}$ low, and DEN goes high to enable the data transceivers.

T_3 – Data phase: $READY$ is sampled, and wait states (T_w) are inserted if it is low. Memory places data on AD15–AD0. The status lines $S_2\overline{S_1}\overline{S_0}$ return to passive (111), which tells the 8288 that the cycle is ending.

T_4 – Data read and end: The CPU reads the data at the start of T_4 . The 8288 returns $\overline{MRDC}$ high and DEN low, and the bus is released.



Maximum mode in 8086

- Bus timing diagram

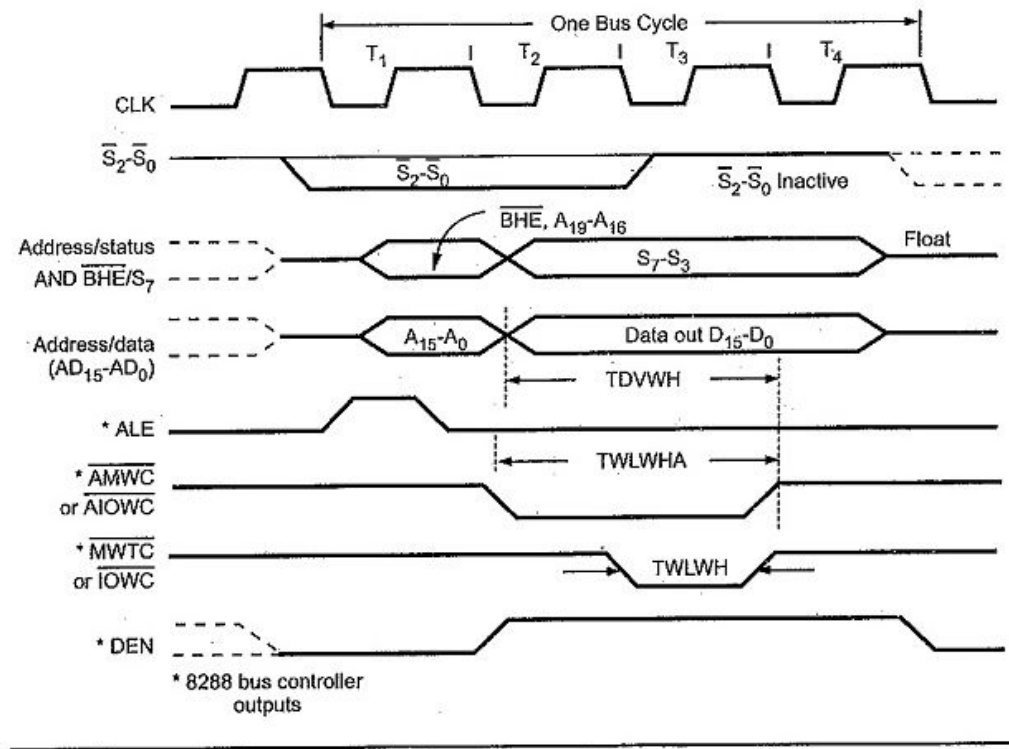
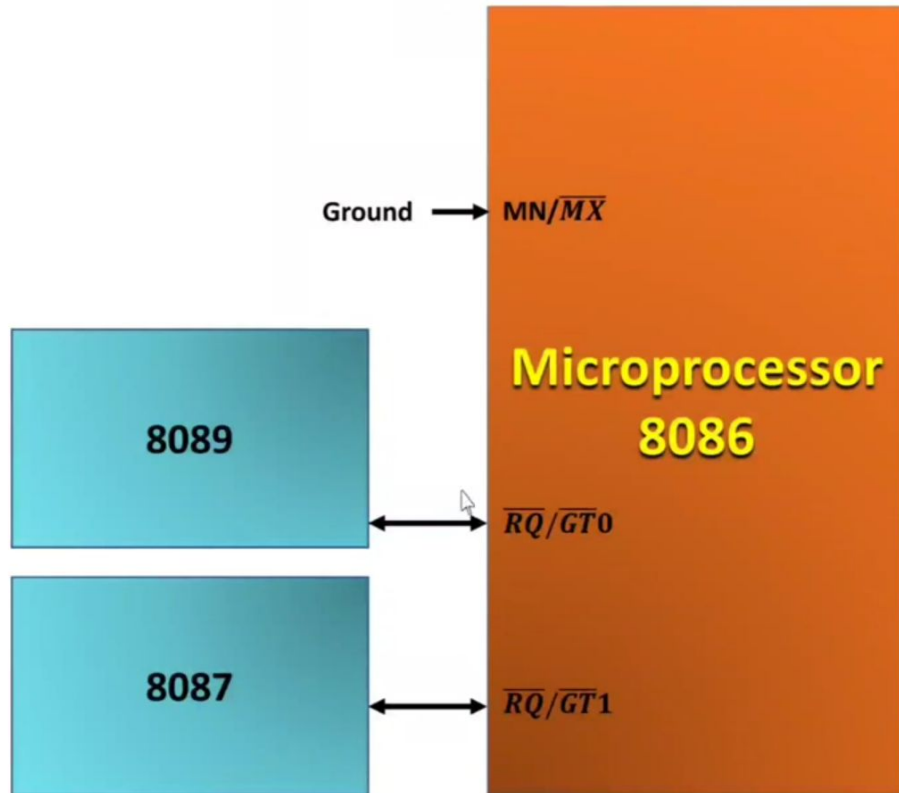


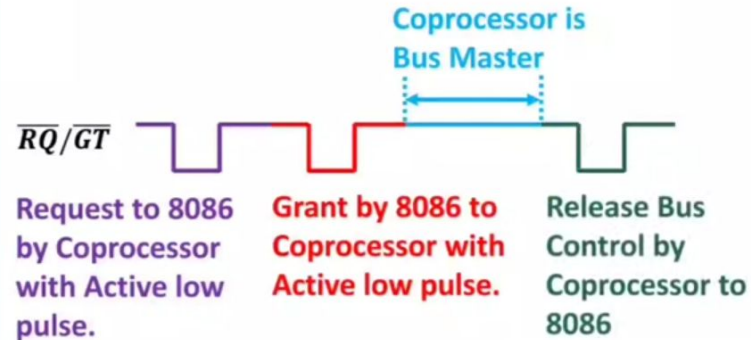
Fig. 10.10 (b) Output (write operation)

1. S_0, S_1, S_2 are set at the beginning of bus cycle. On detecting the change on passive state $S_0 = S_1 = S_2 = 1$, the 8288 bus controller will output a pulse on its ALE and apply a required signal to its DT/R pin during T_1 .
2. In T_2 , 8288 will set $DEN = 1$ thus enabling transceiver. For an input, 8288 it will activates MRDC or IORC. These signals are activated until T_4 . For an output, the AMWC or AIOWC is activated from T_2 to T_4 and MWTC or IOWC is activated from T_3 to T_4 .
3. The status bits S_0 to S_2 remain active until T_3 , and become passive during T_3 and T_4 .
4. If ready input is not activated before T_3 , wait state will be inserted between T_3 and T_4 .

Maximum mode in 8086 - Bus request and bus grant



- ☐ Request/Grant $\overline{RQ}/\overline{GT}$ is used in Maximum mode for coprocessor taking control of system Bus.
- ☐ For Maximum mode, $\overline{MN}/\overline{MX}$ should be connected to ground [logic 0].
- ☐ In Maximum Mode, $\overline{RQ}/\overline{GT}0$ and $\overline{RQ}/\overline{GT}1$ two lines are there for coprocessor operation.
- ☐ $\overline{RQ}/\overline{GT}0$ is having higher priority than $\overline{RQ}/\overline{GT}1$.
- ☐ By default, $\overline{RQ}/\overline{GT}$ lines are there at logic 1.



Maximum mode in 8086 - Lock signal in 8086

